SOFTWARE DESIGN DOCUMENT

Selected Use Case: UC-03 – Generate Personalized Routine

1. System Overview

This document describes the detailed software design for the Generate Personalized Routine use case in the LifeSync system.

The purpose of this use case is to generate personalized diet and exercise plans for users based on their previously determined fitness level. The system uses a Large Language Model (LLM) integration layer to generate customized recommendations.

The design demonstrates how object-oriented design principles and design patterns are applied within the system architecture to ensure modularity, maintainability, and scalability.

2. System Context

The LifeSync system interacts with both internal components and external services.

Primary Actor
User

External System
Large Language Model (LLM) Provider

Interaction Flow
User → Web Application → Backend API → Routine Generation Module → LLM Service → Database → User Dashboard

The system architecture separates responsibilities between frontend, backend business logic, external AI services, and persistent storage.

3. Key Features and Functionality

The Generate Personalized Routine use case provides the following functionality:

- Retrieve the user's fitness level from the database
- Select an appropriate routine generation strategy based on the level
- Construct a structured prompt for the language model
- Send the prompt to the LLM service
- Receive a generated routine from the AI system
- Store the generated routine in the database
- Display the routine in the user dashboard

This functionality allows users to receive personalized health plans automatically generated by AI.

4. Assumptions and Dependencies

The implementation of this use case assumes the following:

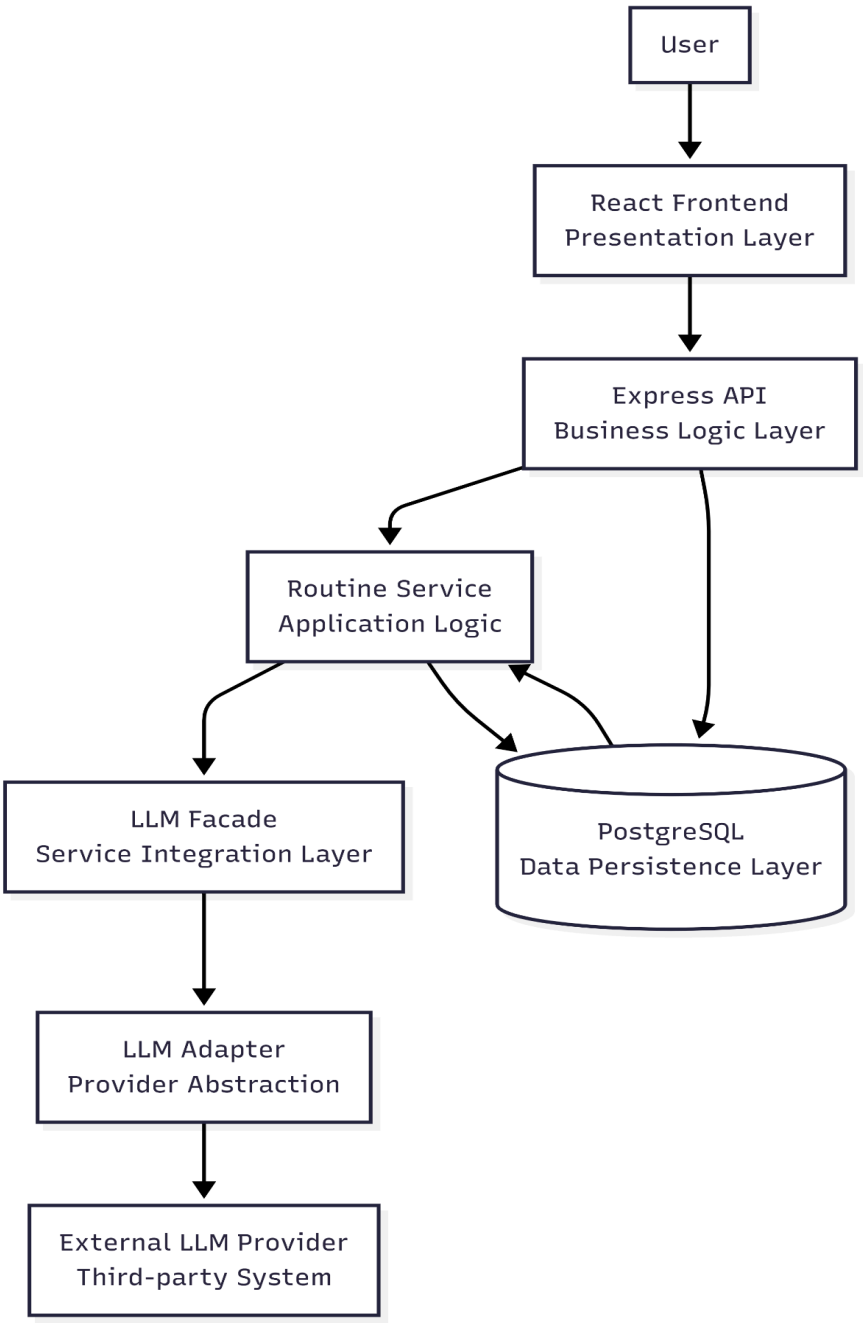
- The onboarding survey has already been completed
- The user's fitness level has already been determined
- The system has access to an external LLM provider
- The database connection is active
- The user is authenticated

Dependencies include:

- Node.js backend environment
- PostgreSQL database
- LLM provider API or local model interface

5. Architectural Design

5.1. System Architecture Diagram (High-Level)



5.2. Architectural Patterns and Styles

The system uses Layered Architecture.

Layers include:

- Presentation Layer
  Handles user interaction and UI rendering.

- Business Logic Layer
  Contains core application logic such as routine generation and user level processing.
- Service Integration Layer
  Manages interaction with external services such as language model providers.
- Data Persistence Layer
  Handles database operations and data storage.

5.3. Rationale for Architectural Decisions

The layered architecture was chosen because it:

- Separates system concerns
- Improves maintainability
- Allows independent modification of components
- Simplifies integration with external services

Additionally, the architecture supports future scalability by allowing the AI integration layer to change independently from the business logic.

6. Component Design

6.1. Subsystems and Modules

The routine generation functionality consists of several main components.

1. RoutineController

Handles incoming API requests from the frontend.

2. RoutineService

Coordinates the workflow for generating routines.

3. RoutineStrategy

Defines algorithms for generating routines based on different user levels.

Concrete strategies include:

- BeginnerRoutineStrategy

- IntermediateRoutineStrategy
- AdvancedRoutineStrategy

4. **LLMFacade (Facade Pattern)**

   Provides a simplified interface to communicate with the LLM provider.

5. **RoutineRepository**

   Handles storing and retrieving routines from the database.

6.2. Responsibilities of Each Component

Component	Responsibility
RoutineController	Receives API requests and sends responses
RoutineService	Orchestrates the routine generation workflow
RoutineStrategy	Provides level-specific routine logic
LLMFacade	Manages interaction with the LLM provider
RoutineRepository	Handles database operations

6.3. Interfaces Between Components

The interaction between components occurs through defined service interfaces.

Example:

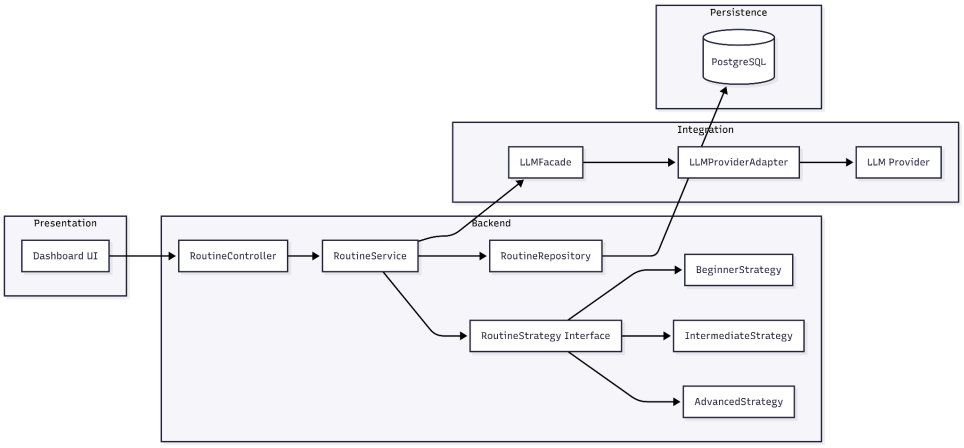
RoutineController → RoutineService.generateRoutine()

RoutineService → RoutineStrategy.generatePrompt()

RoutineService → LLMFacade.generateResponse()

RoutineService → RoutineRepository.saveRoutine()

6.4. Component Diagrams



7. Data Design

7.1. Data Model / ER

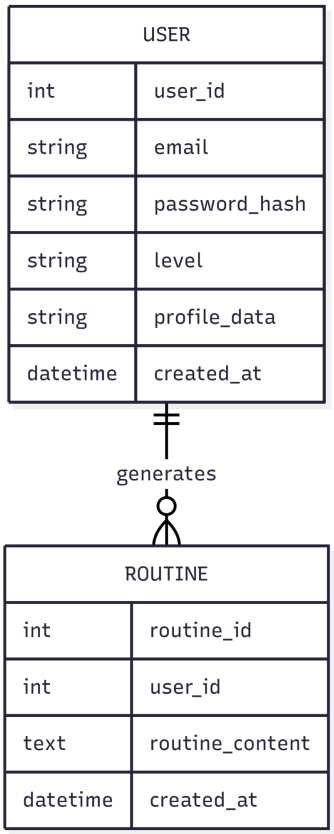
User:

- user_id
- level
- profile_data

Routine:

- routine_id
- user_id
- content
- created_at

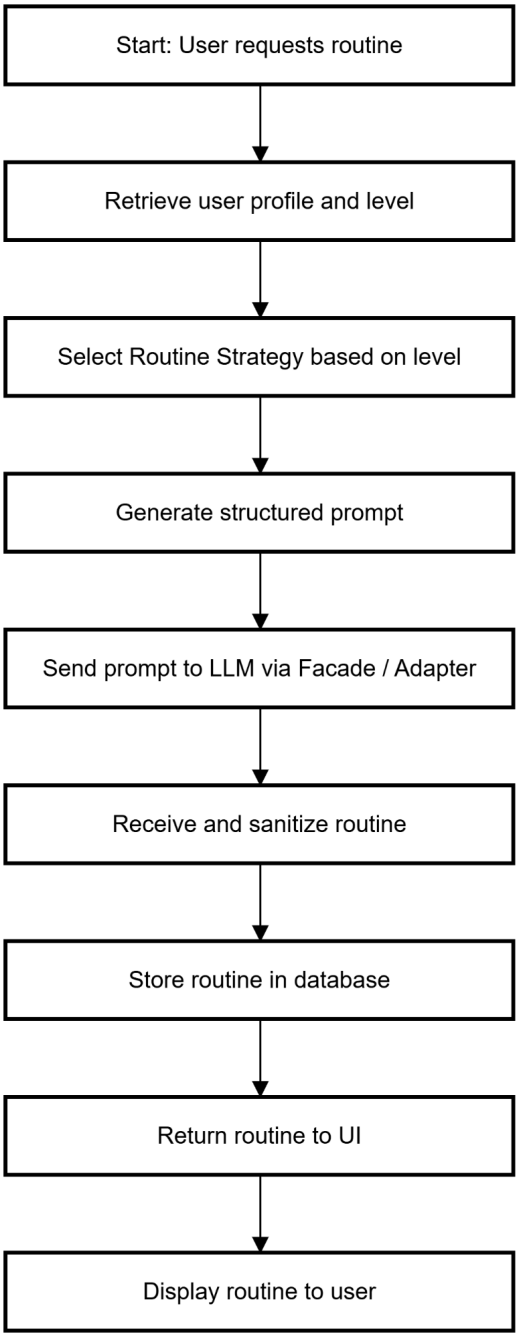
Relationship:
User (1) — (N) Routine



7.2. Data Storage

- The system uses a PostgreSQL relational database.
- Data is stored in structured tables with indexed foreign keys for efficient retrieval.
- Generated routines are stored as structured text content linked to the user.

7.3. Data Flow Diagrams



7.4. Data Validation Rules

The system enforces the following validation rules:

- User must exist in the database
- Fitness level must be valid
- AI response must not be empty
- Generated routines must pass content validation

8. Design Patterns Applied

8.1. Strategy Pattern

Intent:
Allow interchangeable routine-generation logic.

Context:
Different fitness levels require different routine intensity.

Improvement:
New level requires adding a new Strategy class only.

Measurable Benefit:
Reduces conditional branching in core service.

8.2. Facade Pattern

Intent:
Provide simplified interface to LLM interaction.

Context:
LLM communication may involve:

- Token handling
- API formatting
- Response cleaning

Improvement:
Facade isolates complexity from business logic.

8.3. Adapter Pattern

Intent:
Convert external LLM API interface into internal LLMProvider interface.

Context:
Each provider may have different request/response structure.

Improvement:
Switching provider does not affect RoutineService.

Measurable Benefit:
Zero modification required in business logic when changing AI vendor.

9. Implementation Notes

Implementation follows the modular architecture defined in previous documents.

Important considerations include:

- API keys stored using environment variables
- Asynchronous communication with the LLM provider
- Input validation before sending prompts
- Sanitization of AI responses before storing

Code comments are added where design patterns are implemented.

10. User Interface Design

The user interface includes a dashboard where users can generate and view personalized routines.

Main UI elements:

- Generate Routine button
- Routine display section
- Loading indicator while routine is being generated

11. External Interfaces

11.1. APIs

Internal API endpoint:

POST /generate-routine

This endpoint triggers the routine generation process.

11.2 Third-Party Systems

External systems include:

Large Language Model Provider

The system may integrate with different LLM providers through an abstraction layer.

12. Performance Considerations

12.1. Performance Requirements

The system must satisfy the following requirements:

- AI routine generation time ≤ 30 seconds
- Dashboard loading time ≤ 2 seconds

12.2. Scalability and Optimization Strategies

Optimization strategies include:

- Asynchronous API requests
- Database indexing
- Efficient prompt generation
- Optional caching of frequently generated routines

13. Error Handling and Logging

13.1. Exception Management

The system handles the following errors:

- AI service timeout
- Database connection failure
- Invalid user level

Error responses are returned in structured format.

13.2. Logging Mechanisms

The system logs the following events:

- API requests
- AI responses
- System errors

Logs include timestamps and error details for debugging purposes.

14. Design for Testability

The architecture supports testability through modular design.

Testing strategies include:

- Unit tests for strategy classes
- Mock testing for LLM provider
- Integration tests for API endpoints

15. Deployment and Installation

15.1. Environment Configuration

System configuration uses environment variables for:

- Database connection
- LLM provider API keys
- Server port configuration

15.2. Packaging and Dependencies

Required dependencies include:

- Node.js
- Express.js
- PostgreSQL
- Frontend React application

Installation steps:

1. Install dependencies
2. Configure environment variables
3. Start backend server

4. Run frontend application

16. Change Log

Version	Date	Description
1.0	Initial Version	Software design document created

17. Future Work / Open Issues

Future improvements may include:

- Integration with wearable devices
- Adaptive learning based on user progress
- Support for multiple languages
- Advanced analytics dashboard